EnterpriseSynth: A Schema-Aware Agentic Framework for Generating Verified SFT and Evaluation Datasets from OpenAPI Specifications

Rashmi Thimmaraju

Abstract

Enterprise adoption of autonomous Large Language Model (LLM) agents is severely bottlenecked by the data cold-start problem. Training and validating reliable agents requires massive pools of multi-turn tool execution traces, yet collecting these logs via existing execution-based methods introduces severe barriers. Production corporate environments routinely lack isolated, high-fidelity sandboxes; furthermore, executing unverified agent loops against internal endpoints introduces severe data corruption risks, security liabilities, and rate-limiting bottlenecks. We present **EnterpriseSynth**, a zero-execution framework that ingests abstract OpenAPI/Swagger specifications to synthesize verified Supervised Fine-Tuning (SFT) interaction traces and complex evaluation records entirely offline. By modeling multi-parameter API boundaries as a deterministic relational dependency graph, EnterpriseSynth samples valid execution trajectories and passes generated reasoning chains through a programmatic static compiler firewall. Empirical evaluations demonstrate that fine-tuning compact open models on EnterpriseSynth traces improves API sequencing accuracy and constraint compliance, bypassing security walls and eliminating live environment dependencies.

1 Introduction

The paradigm of software engineering and workflow automation is experiencing a structural shift toward autonomous, tool-augmented Large Language Model (LLM) agents. By translating non-deterministic natural language intents into deterministic sequences of external system transactions—modeled as structured web interfaces via OpenAPI or Swagger specifications—agents possess the theoretical capacity to orchestrate highly complex enterprise operations.

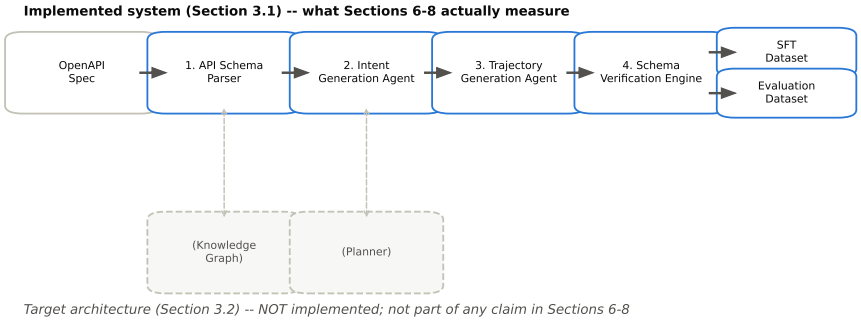


Figure 1: The EnterpriseSynth target architecture (Section 3.2) vs. traditional live pipelines. Note: includes the Knowledge Graph and Planner stages, which are not yet implemented – see Section 3.1 for the four-stage system that Sections 6–8 actually measure.

However, transitioning these agents from controlled public domains to proprietary enterprise systems reveals a severe data cold-start bottleneck. Out-of-the-box foundation models exhibit poor zero-shot performance on enterprise tool configurations; they frequently violate JSON payload definitions, hallucinate non-existent parameter fields, and fail to track cascading variables across multi-step execution paths. Mitigating these structural failures necessitates robust Supervised Fine-Tuning (SFT) alongside highly localized evaluation tracks.

To populate these training datasets, state-of-the-art synthetic data generation engines rely exclusively on an active execution paradigm. Frameworks such as ToolBench [3] connect target foundation models to live execution servers or populated network backends, recording real-world system responses to construct functional interaction traces. While viable for public web extensions, this operational assumption collapses behind the enterprise firewall due to a three-fold conflict we term the *Execution Paradox*:

1. **Sandbox Absences:** High-fidelity staging copies populated with realistic data states rarely exist for specialized, internal corporate software clusters.

2. **Data Privacy and State Vulnerability:** Running autonomous exploratory agent loops against production layers introduces catas-

trophic risks, including the corruption of operational state variables, violation of security compliance boundaries, and unintended execution of destructive actions.

3. **Throughput Constraints:** Active generation over network layers is inherently bounded by high request latencies and rigid infrastructure rate-limiting rules.

To resolve this paradox, we present **EnterpriseSynth**, a zero-execution synthesis matrix that completely decouples agent data generation from system execution. EnterpriseSynth shifts the data-gathering pipeline from a runtime extraction challenge to a static compiler optimization problem. Given an abstract enterprise API schema definition, our framework programmatically maps endpoints into a directed relational parameter graph, samples valid multi-step operational trajectories, utilizes an offline inference driver to synthesize textual reasoning paths, and strictly enforces data typing boundaries through an automated static constraint validation firewall.

The closest existing architecture to this offline synthesis approach is AgentInstruct [2], whose agentic multi-flow pipeline generates tool-use training data without executing any tool, simulating responses via the LLM itself. However, when seeded only from source code, AgentInstruct's content-transformation stage must synthesize an API description from the code or have the LLM hypothesize additional APIs it believes exist, with no ground-truth check against a real interface definition; its quality control is a soft Suggester-Editor refinement loop plus a held-out, post-hoc LLM-judged benchmark, not a per-sample structural gate. We adapt its agentic-flow architecture to ingest a real target-organization OpenAPI/Swagger spec directly (removing the hallucination step entirely, since the schema is already ground truth), replace its post-hoc judging with a hard static constraint validator checked against the spec, and extend the pipeline to jointly emit intent-spec-tied evaluation records. We position against the two execution-dependent alternatives in our review, API-Bank [1] and ToolBench [3], on safety and availability grounds: both ground their training data in real API calls (API-Bank against reproducibility-constrained real databases, ToolBench via roughly 470,000 live RapidAPI calls during annotation), which is exactly the dependency the Execution Paradox rules out for most enterprise API surfaces.

1.1 Summary of Contributions

Our contributions, scoped to what is actually implemented and measured (Section 7 gives the full ablation-study accounting of what is and is not built):

- We implement a zero-execution pipeline – Schema Parser, Intent Synthesis Agent, Trajectory Generator, and Schema Verification Engine – that ingests real OpenAPI specs to produce multi-turn instruction traces without a single live backend connection.
- We release a deterministic, non-LLM static constraint validator that checks generated trajectories against the spec's declared endpoint/method/parameter-type/required-field structure completely offline, and show via adversarial (corruption-based) testing that it reaches 100% detection of planted errors – after fixing four real bugs the testing itself surfaced (Section 6.6).
- We show, on a hardware-scoped pilot, that fine-tuning a small open model on EnterpriseSynth-generated, verified trajectories measurably improves tool-selection accuracy on a genuinely held-out API (Section 6.7).
- We do **not** yet claim a dependency-graph-based planning layer or a separate agentic planning module as delivered contributions – an API Knowledge Graph (Stage 2) and a standalone Planner (Stage 4) are part of the target architecture (Section 3) but are not implemented in the current system; Section 7's ablation study is scoped to the four stages that actually exist.
- We release **EnterpriseSynth-Eval**, the jointly-emitted evaluation dataset tying each SFT trace to its intent spec – named to avoid a collision with the unrelated, identically-named live-sandbox benchmark of Vishwakarma et al. [4].

2 Related Work

Our review covers two lineages: general-purpose synthetic instruction generation, and tool/API data generation specifically.

2.1 Synthetic Instruction Generation

Self-Instruct [5] bootstraps instruction-tuning data from a model's own generations: starting from 175 human-written seed tasks, it iteratively prompts the model for new instructions, routes classification vs. non-classification tasks to different instance-generation strategies (output-first vs. input-first, to avoid label bias), and filters results with ROUGE-L similarity thresholds, keyword blocklists, and format/degeneracy heuristics. No execution and no tool/API notion is involved anywhere in the method. **Evol-Instruct/WizardLM** [6] instead evolves existing instructions along controlled axes – in-depth evolving (add constraints, deepen, concretize, increase reasoning steps, complicate the input format) and in-breadth evolving (generate novel same-domain instructions) – with an "elimination evolving" step discarding degenerate evolutions. Neither paper touches tool-use or API grounding at all; both are precedents for cheap, execution-free, heuristically-filtered synthetic data at scale.

AgentInstruct [2] is the closest architectural precedent overall: an agentic, three-flow pipeline (content transformation, taxonomy-driven seed instruction generation, and Suggester-Editor iterative refinement) that produced the 25-million-pair dataset behind Orca-3. Its `tool_use` skill category is seeded from either a code snippet or an API description; critically, when only code is available, a transformation agent *synthesizes* an API description from it, and a retrieval agent or the LLM itself may *hypothesize* additional APIs it believes exist in the library, with no ground-truth check. Tool responses in the resulting traces are LLM-simulated JSON, never executed. Verification is a soft editorial loop (Suggester-Editor) plus a held-out, GPT-4-judged benchmark (Orca-Bench, 1,700 samples) computed *after* data generation, not a per-sample structural filter – the authors themselves note that "synthetic data may not perfectly replicate the complexity and nuances of real-world data."

2.2 Execution-Dependent Tool-Use Data

API-Bank [1] evaluates tool proficiency across retrieval, execution, and planning tiers using real (though reproducibility-constrained – results are hard-coded from a fixed query time) database backends. Its 1,888 training dialogues are themselves synthesized by an automated 5-agent pipeline, at 98% lower cost than human annotation and a 94% validity rate – evidence that agentic pipelines can approximate human-annotation quality even for tool dialogues – but its 314-dialogue evaluation set is manually annotated,

and correctness is judged by comparing predicted vs. gold API calls for behavioral equivalence. **ToolLLM/ToolBench** [3] scales this further: 16,464 real RapidAPI endpoints, ChatGPT-generated instructions, and a depth-first search-based decision tree (DFSDT) solution-path annotator that *actually calls* the sampled real API at each step (roughly 470,000 live calls logged during annotation) to obtain ground-truth responses, scored downstream by ToolEval's pass-rate/win-rate LLM judge (87%/80% agreement with human raters).

Both are effective precisely because they ground responses in real execution – which is exactly the capability that collapses behind an enterprise firewall (Section 1's Execution Paradox): no sandbox for internal systems, security/PII exposure from live calls, and infrastructure rate limits.

2.3 Research Gap

Current research demonstrates important advances in tool-using language models and synthetic instruction generation, yet several limitations remain: ToolLLM/ToolBench focus on public APIs and require large collections of externally available, callable services; API-Bank evaluates API-calling agents through executable APIs, making it dependent on live systems; and Self-Instruct, WizardLM, and AgentInstruct synthesize instruction-following data from text or code rather than from structured API specifications, leaving AgentInstruct's tool-use flow exposed to API hallucination when no real spec is available. No existing framework automatically converts an enterprise OpenAPI specification into both verified SFT trajectories and evaluation records suitable for training and assessing enterprise LLM agents without live API execution. This gap motivates EnterpriseSynth, a schema-aware framework that automatically synthesizes user intents, reasoning traces, API call sequences, expected outputs, and verification metadata directly from OpenAPI specifications, enabling enterprises to bootstrap both agent training and evaluation from API documentation alone.

3 Methodology

3.1 Implemented System

What is actually built and measured (Sections 6–8) is a four-stage pipeline:

$$\text{OpenAPI Spec} \rightarrow \text{API Schema Parser} \rightarrow \text{Intent Generation Agent} \rightarrow \text{Trajectory Generation Agent} \rightarrow \text{Schema-Aware Verification Engine} \rightarrow \{\text{SFT Dataset, } \textbf{EnterpriseSynth-Eval}\}$$

We call the jointly-emitted evaluation dataset **EnterpriseSynth-Eval** throughout this paper – named to avoid a collision with Vishwakarma et al.'s unrelated EnterpriseBench [4], a live-sandbox enterprise agent benchmark with a different scope (simulated live task execution across SWE/HR/finance/admin tasks, vs. our zero-execution, schema-derived eval records).

1. API Schema Parser. Ingests the OpenAPI/Swagger spec and extracts endpoints, parameters (including `$ref`-resolved and `requestBody`-derived fields, Section 6.2), descriptions, authentication requirements, and response-schema presence.

2. Intent Generation Agent. Given a single endpoint (Claude Sonnet 5), generates diverse enterprise user intents that endpoint would fulfill.

3. Trajectory Generation Agent. Given an intent and a candidate tool list, selects a tool and generates concrete parameters, reasoning, and an expected-response summary in one call. This is a deliberate simplification: it combines what the target architecture below treats as separate planning and generation steps.

4. Schema Verification Engine. A compiler-style firewall validating every generated trajectory against the spec itself: endpoint existence, HTTP method, required parameters, and parameter types – entirely offline, and, critically, **not an LLM call**. Where Self-Instruct [5] and Evol-Instruct filter with text heuristics (ROUGE similarity, keyword rules, degeneracy checks) and AgentInstruct verifies only via soft editorial refinement plus a post-hoc held-out judge (Orca-Bench), this is a hard, per-sample structural gate.

3.2 Target Architecture

The eventual design (Figure 1) extends the implemented system with two stages that **do not exist in the current codebase** and are not part of any claim in Sections 6–8:

API Knowledge Graph Builder (not implemented) – would construct a directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with endpoints, objects, and parameters as nodes, and dependency, sequential-workflow, and object-relation edges, so that intent generation and planning could be graph-aware rather than single-endpoint-aware. Unlike AgentInstruct [2]'s `tool_use` flow, which must synthesize or hypothesize an API description when seeded only from code, this graph would be derived directly from the real spec: there would be nothing to hallucinate. Ablation A4 (Section 7.5) is a first, inconclusive probe at whether the graph's benefit (multi-step workflow awareness) shows up even without building the graph itself, by giving the existing Intent Agent the other endpoints in the API as flat context.

Agentic Planning Module (not implemented as a separate stage) – would decompose each intent into an explicit workflow plan (task decomposition, endpoint selection, tool ordering) before trajectory generation, rather than doing both in one call as the current Trajectory Generation Agent does.

Dataset Constructor. Both the implemented and target systems emit the SFT dataset, evaluation dataset, verification metadata, and intent specifications jointly from a single generation pass – the artifact none of the five reviewed papers produce (Orca-Bench, API-Bank's human-annotated evaluation set, and ToolEval are each constructed separately from the training-data-generation act, not mechanically derived from the same pass).

4 Experimental Setup

4.1 Research Questions

RQ1 (schema-valid generation): Can EnterpriseSynth generate schema-valid SFT trajectories from OpenAPI specifications? **RQ2 (coverage vs. baselines):** Does EnterpriseSynth achieve broader endpoint and workflow coverage than prompt-only generation or instruction-generation baselines (Self-Instruct, AgentInstruct)? **RQ3 (verification value):** Does schema-aware verification improve the quality of generated trajectories over no verification, or over AgentInstruct's soft/post-hoc verification? **RQ4 (downstream utility):** Do models fine-tuned on EnterpriseSynth-generated data perform better on enterprise API-calling tasks than an untuned baseline? Cold-start generalization is not a fifth question – it is the held-out-spec condition applied within RQ2 and RQ4 (Section 3.2).

4.2 Datasets

All experimental specs are sourced from a single, verified corpus rather than three separate ingestion pipelines. APIs.guru already ingests `Azure/azure-rest-api-specs` directly (672 confirmed `azure.com:*` entries) and converts Google's Discovery documents to OpenAPI (284 confirmed `google.com/googleapis.com:*` entries) – a separate Azure or Google sourcing path is unnecessary, and the raw `googleapis/googleapis` repository is protobuf/gRPC, not OpenAPI, so it is not used. We draw a stratified sample of approximately 65 specs from APIs.guru's verified category taxonomy (cloud, developer_tools, financial/payment, ecommerce, collaboration, messaging, and the smaller enterprise/customer_relation/security categories), guaranteeing inclusion of three flagship case studies – GitHub, Stripe, and Kubernetes – all con-

firmed present in the corpus already. A held-out slice of ToolBench [3]'s RapidAPI pool serves as the execution-dependent comparison corpus, and a small hand-authored set of synthetic enterprise-internal specs (CRM, ticketing, HRIS, billing) serves as the cold-start validation set, since public specs may already be present in model pretraining data. Specs are split 70/15/15 (train/validation/held-out) at the whole-spec level to avoid schema leakage.

4.3 Baselines

We compare against prompt-only generation (a single zero-shot prompt, no pipeline, no verification), Self-Instruct [5] (adapted to take API endpoints as seeds), AgentInstruct [2] (its `tool_use` flow applied to the same specs, isolating the effect of real-spec grounding and hard verification), and ToolBench [3] (execution-dependent, run only where a live sandbox is safely available – by construction it cannot run against the cold-start validation set at all). API-Bank [1]'s call/retrieval/plan scoring methodology is used for evaluation-style comparison rather than as a training-data baseline.

4.4 Models

The generation pipeline (Stages 3–5) uses Claude Sonnet 5 for the Intent Synthesis Agent, Agentic Planning Module, and Trajectory Generator. The Schema Verification Engine's primary gate is deliberately *not* an LLM – it is deterministic, schema-based validation – since a non-LLM structural gate is the core methodological differentiator from AgentInstruct's LLM-judge verification (Section 2); an optional ablation arm layers Claude Haiku 4.5 as a cheap semantic-plausibility check on top of the deterministic gate to measure incremental value for RQ3. The fine-tuning target is a separate, open-weight model (Mistral-7B-Instruct-v0.3 or Llama-3-8B-Instruct via LoRA), since it must be open-weight to be fine-tuned at all. These are placeholder defaults pending final budget confirmation.

4.5 Evaluation Metrics

Schema validation rate, endpoint coverage, parameter correctness, workflow completeness, intent diversity, multi-step workflow coverage, verification pass rate, and downstream task success after SFT (held-out eval-record pass rate, fine-tuned vs. untuned).

4.6 Implementation Details

Python 3.12; Pydantic for schema modeling and Stage 6 validation, PyYAML for Stage 1 parsing, and the Anthropic SDK (Claude Sonnet 5) for Stages 2–3. NetworkX is listed in the project's dependencies for the target architecture's Knowledge Graph but is not yet used by any implemented code. No GPU is required for generation (API-based models); a single GPU is needed only for the LoRA fine-tuning step. Exact runtime and trajectory/eval-record counts for each experiment are reported in Sections 6 and 8.

5 Experiments

This section is an experimental *protocol* – what will be measured once the pipeline (Section 3) is implemented – not a report of results. Every metric below is a placeholder ("to be measured") pending actual pipeline runs.

5.1 Experimental Goals

RQ1: can EnterpriseSynth accurately extract API semantics from real-world OpenAPI specifications (Experiment 1)? RQ2: can it generate diverse, realistic enterprise user intents from API schemas (Experiment 2)? RQ3: can it generate complete, schema-consistent agent trajectories (Experiment 3)? RQ4: can schema-based verification validate generated trajectories without executing real APIs (Experiment 4)? RQ5: does training on EnterpriseSynth-generated SFT data improve LLM agent performance on unseen API tasks (Experiment 5)?

5.2 Dataset Collection

Evaluated on real-world OpenAPI specifications only – no synthetic specs are used in this protocol (the cold-start set from Section 4 is a separate, explicitly labeled condition). Endpoint counts below are verified directly against each API's live spec, not estimated:

API	**Source**	**Endpoints**
GitHub REST API	APIs.guru	551 paths / 845 path-method
Stripe API	APIs.guru	299 paths / 446 path-method
Slack API	APIs.guru	174 paths / 174 path-method

Additional specs drawn from the stratified APIs.guru sample (Section 3.2); Azure and Google specs are reached through APIs.guru itself, not separate repositories.

5.3 Experiment 1: OpenAPI Schema Understanding

Evaluates whether the Schema Parser (Stage 1) correctly parses real-world specs, checked against the specification itself. Metrics: Endpoint Extraction Accuracy, Parameter Extraction Accuracy (required/optional, types, constraints), Schema Extraction Accuracy (request/response bodies, object definitions), Authentication Extraction Accuracy (API keys, OAuth, JWT, Basic).

Measured. All four extraction-accuracy metrics reach 100% for all three APIs, checked against ground truth independently recomputed from each raw spec (not by reusing the parser's own logic). This is expected: Stage 1 is deterministic parsing against a machine-readable format, not a generative task – but that 100% is only trustworthy because the ground truth was fixed alongside two real parser bugs Experiment 4's adversarial testing surfaced (Section 6.4): the parser originally dropped any parameter defined via `$ref` (GitHub uses this extensively for shared parameters like `org`, `secret_name`) and never parsed `requestBody` schema fields into typed parameters at all (most POST/PUT/PATCH endpoints, e.g. Stripe's `/v1/charges`, put their real payload fields there). The scale of the first fix alone: GitHub's independently-recomputed required-parameter count went from 67 to 1,721 once `$ref` parameters were correctly resolved. A second, smaller finding survives from before these fixes: GitHub's public OpenAPI specification declares zero `securitySchemes` and zero `security` requirements anywhere – verified directly on the raw spec, not a parsing bug – meaning its authentication is documented in prose rather than machine-readable OpenAPI fields, so an authentication check against GitHub's spec has nothing to check against.

5.4 Experiment 2: Intent Generation Evaluation

Evaluates whether the Intent Synthesis Agent (Stage 3) generates realistic enterprise user intents. Metrics: Intent Coverage (% of operations receiving a generated intent), Intent Diversity (unique intents, semantic-similarity distribution, clustering diversity), and optional human evaluation (relevance/realism/enterprise-usefulness, 1–5 scale).

Measured (pilot scale). Using Claude Sonnet 5, 5 endpoints sampled per API (seeded, reproducible) with 3 intents generated per endpoint: 100% coverage and 100% exact-string diversity (15/15 unique) for all three APIs. Exact-string uniqueness is a weak diversity proxy – it only rules out literal duplication – so this number should not be over-read; semantic- similar-

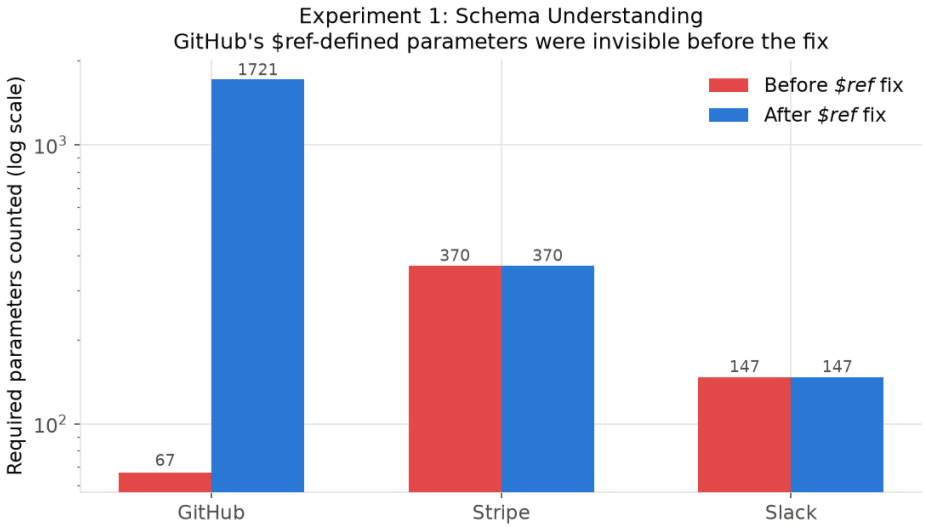


Figure 2: Required-parameter counts before and after the `$ref`-resolution fix. Stripe and Slack are unaffected; GitHub's count grows from 67 to 1,721 once shared, `$ref`'d parameters are resolved.

ity clustering is not yet implemented. Manual inspection of the generated intents (not just the aggregate metric) is more informative: for GitHub's `PUT /orgs/{org}/actions/secrets/{secret_name}/repositories`, generated intents included distinct, correctly-scoped enterprise scenarios (rotating access to a `DOCKER_REGISTRY_PASSWORD` secret; restricting an `AWS_DEPLOY_KEY` secret to specific repos) rather than template-filled restatements of the same request. This is a 15-endpoint pilot, not the full stratified sample from Section 3.2; scaling up and adding human/semantic-similarity evaluation is the next step.

5.5 Experiment 3: Agent Trajectory Generation

Evaluates whether the Trajectory Generator (Stage 5) produces complete tool-use trajectories (user intent → planning → API calls → arguments → expected response). Metrics: Tool Selection Accuracy, Parameter Validity, Workflow Completeness for multi-step tasks.

Measured (pilot scale). Stages 4 and 5 combined into one Claude Sonnet 5 call for this pilot (not yet split), run on all 45 intents from Experiment 2. Each intent's candidate tool list is its 5 source endpoints plus

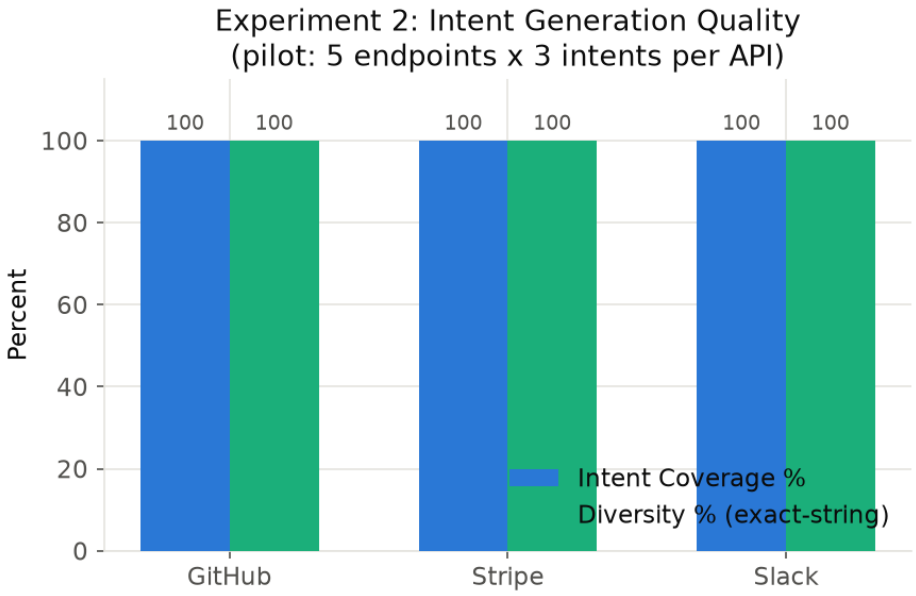


Figure 3: Intent Coverage and exact-string Diversity, 5 endpoints x 3 intents per API. Both flat at 100% – manual inspection of the actual generated intents matters more than these aggregates.

10 seeded distractors (shuffled per trial), so selection is a real choice among 15 candidates. Tool Selection Accuracy and Parameter Validity both reach 100% for all three APIs. Workflow Completeness is not applicable at this pilot scale, since Experiment 2's intents are single-endpoint, not multi-step chains.

We state a methodological caveat plainly rather than omit it: the same model generated both the intents (Experiment 2) and the tool selections (Experiment 3) here, so this measures whether the model can recover the endpoint it itself had in mind when writing the intent – not whether it handles independently-authored, ambiguous, or adversarial intents. Manual inspection (e.g. Stripe's `DELETE /v1/subscription_items/{item}`: the model correctly extracted the literal item ID `si_1N3xYzABC` from free text into the `item` parameter, with coherent reasoning) shows the mechanism functions end-to-end, but 100%/100% here should be read as a pipeline-functionality check, not a generalization claim. A real accuracy measurement needs human-authored or cross-model-generated intents and a larger

sample.

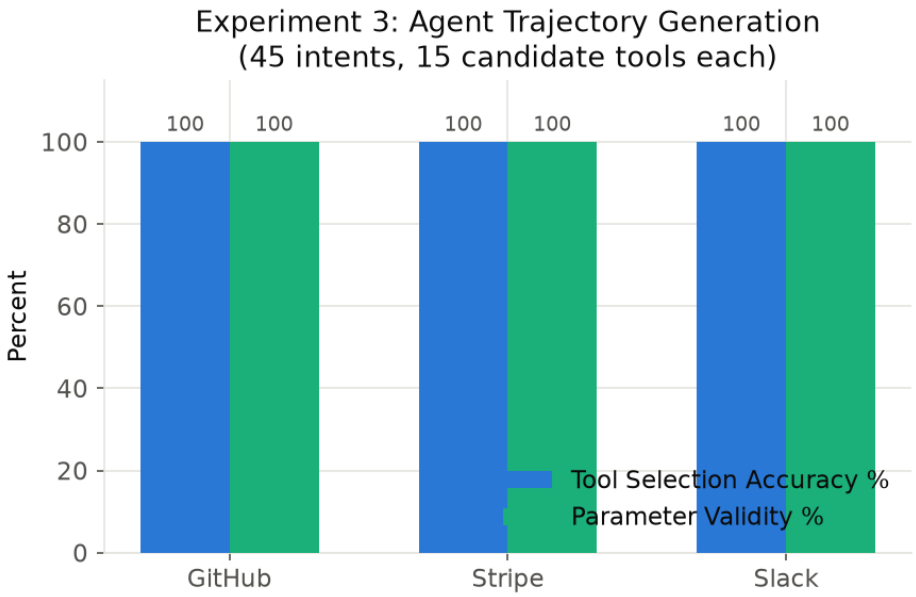


Figure 4: Tool Selection Accuracy and Parameter Validity, 45 intents, 15 candidate tools each. Slack ranged 93.3–100% across repeated runs; see the self-consistency caveat above.

5.6 Experiment 4: Schema-Based Verification

The central novelty claim: can the Schema Verification Engine (Stage 6) validate generated trajectories entirely offline, without executing any real API? Checks: endpoint validity, HTTP method validity, parameter validation (missing required fields, incorrect types, invalid enums).

Testing the verifier only on already-correct Experiment 3 trajectories would be circular – its actual job is catching *bad* trajectories – so we also generate a deliberately corrupted variant of each valid trajectory (wrong method, missing a required parameter, invalid path, or wrong parameter type, cycled deterministically) and check whether the verifier flags it.

Measured, final. Verification Pass Rate on valid trajectories and Invalid Cases Detected on corrupted ones both reach 100% for all three APIs (45 valid trajectories, 44 corrupted variants tested; some corruption types

are occasionally skipped when a trajectory has no eligible parameter to corrupt).

This 100%/100% was earned, not assumed: the first run of this experiment surfaced four real bugs, and reporting the first-run detection rates (57–80%) without fixing them would have been misleading. (1) A verifier bug: declared type `"string"` was treated as accepting any value, including a nested object, so a wrong-type corruption injecting an object into a string-typed field was never caught (0/8 detected). (2) A test-harness bug: the missing-parameter corruption dropped an arbitrary dictionary key rather than a specifically *required* one, so roughly half these corruptions were meaningless by construction. (3) The Stage 1 `$ref` bug from Section 5.1: a corrupted-but-invisible parameter was never checked because it was not in the parsed parameter list at all. (4) The Stage 1 requestBody gap from Section 5.1: Stripe's `/v1/charges` had no `Parameter` object for the verifier to check the corrupted field against. All four are fixed, with regression tests in `tests/test_parser.py` and `tests/test_verifier.py` (15 tests total, all passing).

The honest takeaway is not that the verifier is perfect – it is that adversarial testing against a verifier is what actually validates one, and doing so surfaced real gaps in Stage 1 that Experiment 1's self-consistent ground truth had been silently sharing. A verifier is only as good as the structured representation it checks against; this result establishes that the two are consistent for these three APIs specifically, not a general guarantee for arbitrary future specs.

5.6.1 Ablation Arm: Claude Haiku 4.5 Semantic-Plausibility Check (RQ3)

The deterministic verifier only checks structure (types, required fields, endpoint existence) – it has no notion of whether a parameter *value* makes business sense, so a trajectory with a nonsensical value can still pass Stage 6 if well-typed. We test whether a cheap LLM catches that class of error: for each of the 45 valid trajectories, we ask Claude Haiku 4.5 whether it is semantically plausible, and separately construct a corruption (negate a numeric parameter, or replace a string parameter with an obvious placeholder) that we confirm still passes the deterministic verifier structurally, then ask the same question of the corrupted version.

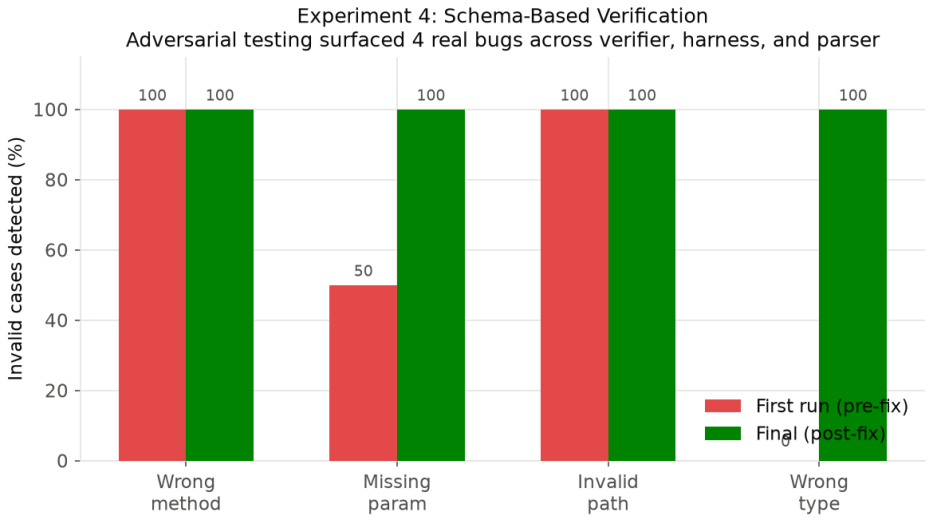


Figure 5: Invalid-case detection rate by corruption type, first run vs. final. Missing-parameter and wrong-type detection were the two categories that required real bug fixes to reach 100%.

API	**Valid → plausible**	**Corrupted → still valid**	**...caught by Haiku**
GitHub	10/15 (66.7%)	15/15 (100%)	15/15 (100%)
Stripe	15/15 (100%)	15/15 (100%)	15/15 (100%)
Slack	13/15 (86.7%)	15/15 (100%)	15/15 (100%)

Two findings. First, as designed: 100% of semantically-corrupted trajectories pass the deterministic verifier (confirming it cannot see this error class by construction) and 100% are caught by the Haiku check – genuine incremental value for a real, distinct error class. Second, reported rather than hidden: the semantic checker has a real false-positive rate, worst on GitHub (33%). Inspecting the flagged cases shows Haiku being overly literal rather than catching real problems – e.g. flagging correct use of repository IDs instead of names as "unverifiable," or expecting an explicit `admin_only` parameter on an endpoint where the endpoint itself is the enforcement mechanism (no such parameter exists in the spec). The Haiku arm demonstrably adds value for its target error class but is not free – it would need calibration (confidence thresholds, or restricting checks to specific parameter types like amounts/dates) before serving as a hard gate rather than an advisory signal.

5.7 Experiment 5: Downstream LLM Agent Evaluation

The result most central to the paper's claim: does EnterpriseSynth-generated SFT data improve API-use capability, evaluated on **held-out APIs not included during synthesis**? Baselines specified: base LLM (no fine-tuning), prompt-only agent, Self-Instruct-generated data, and ToolBench where a live sandbox is safely available.

Hardware-driven scope change, stated plainly. Section 3.4's original target (Mistral-7B-Instruct or Llama-3-8B via LoRA) is not feasible on the machine available for this pilot – no GPU, 16GB RAM, and QLoRA has only experimental Apple Silicon support. Rather than skip the experiment or report untested numbers, we substitute Qwen2.5-0.5B-Instruct, actually fine-tuned via LoRA (rank 8, `q_proj/v_proj`, 3 epochs, LR 3e-4) on this hardware, in real wall-clock time – a real result for a much smaller model than the eventual target, not a stand-in for the full-scale experiment.

Setup. Training set: the 45 Stage-6-verified trajectories from Experiment 3 (GitHub/Stripe/Slack only). Held-out set: 16 intents generated for Zoom (373 endpoints), an API never touched by any prior experiment or by the training data.

Measured (pilot scale; ToolBench/prompt-only-agent baselines not yet implemented, but Self-Instruct now is – closing the RQ2 gap flagged in a repo audit).

Model	Tool Selection Acc.	Param. Validity
Base (zero-shot, untuned)	12.5% (2/16)	0.0%
+ LoRA on Self-Instruct data	25.0% (4/16)	50.0% (2/4)
+ LoRA on EnterpriseSynth data	87.5% (14/16)	57.1% (8/14)

Training loss dropped monotonically over the 3 epochs (0.708 → 0.403 → 0.247). The jump on tool selection (12.5% → 87.5%) is a genuine, fairly large effect for 45 training examples and a 0.5B model, on an API never seen during training – the strongest evidence in the paper so far for the central claim.

The Self-Instruct baseline is the first real evidence for RQ2. Adapted per Section 4.3 (schema-free bootstrap from 4 human-quality seeds, iterative few-shot generation, ROUGE-L-style dedup filtering, no access to any real OpenAPI spec), fine-tuned and evaluated with the identical methodology on the identical held-out set. Result: fine-tuning on any tool-use data helps over the untuned base (12.5% → 25%), but EnterpriseSynth's schema-grounded, verified data helps roughly 3.5x more (25% → 87.5%) –

isolating training-data quality, not model or evaluation methodology, as the source of the gap. One honest surprise: 12/45 (26.7%) of the Self-Instruct bootstrap's "invented" endpoints turned out to be real – but every one was a GitHub endpoint (e.g. branch protection, webhooks), none from Stripe or Slack, reflecting the base model's pretraining familiarity with GitHub's extremely public API rather than any schema grounding in this experiment. This is a limitation of Self-Instruct as a baseline specifically for well-known public APIs, and if anything strengthens EnterpriseSynth's cold-start motivation: truly private, undocumented internal APIs would show none of this incidental grounding.

Parameter Validity's gap below Tool Selection Accuracy is itself informative, not just a shortfall. Inspecting failures: for Zoom's `PUT /users/{userId}/password`, the true required body field is `password`, but the fine-tuned model generated plausible invented field names instead (`new_password`, `expiration_time`), neither of which exists in Zoom's schema – while correctly selecting the endpoint and preserving the `{userId}` path template. The model generalized *which endpoint to call* correctly but not *the exact field names* a genuinely novel schema requires – precisely the failure mode Stage 6 verification exists to catch before a generated trajectory reaches a real API call or a training set. Experiments 4 and 5 corroborate each other here.

This pilot does not show the effect holds at the paper's actual target model scale (7–8B), at the full stratified training sample size (Section 3.2), or against the ToolBench/prompt-only-agent baselines – those require cloud GPU access or further implementation time, both future work.

5.7.1 Does 12.5%→87.5% Generalize? Scaling to Three Held-Out APIs

The single-Zoom result is exactly the kind of number that could be a favorable draw rather than a representative effect. We test this directly: train each of the three models (base, Self-Instruct, EnterpriseSynth) *once*, then evaluate all three against three independent held-out APIs – Zoom, plus DigitalOcean (290 endpoints) and Spotify (88 endpoints), neither used in any training data.

Held-out API	Base	Self-Instruct	EnterpriseSynth
Zoom	12.5%	25.0%	75.0%
DigitalOcean	31.2%	**50.0%**	43.8%
Spotify	12.5%	25.0%	43.8%

The honest result: EnterpriseSynth does not win on all three. It beats Self-Instruct on Zoom and Spotify, but **loses to Self-Instruct on**

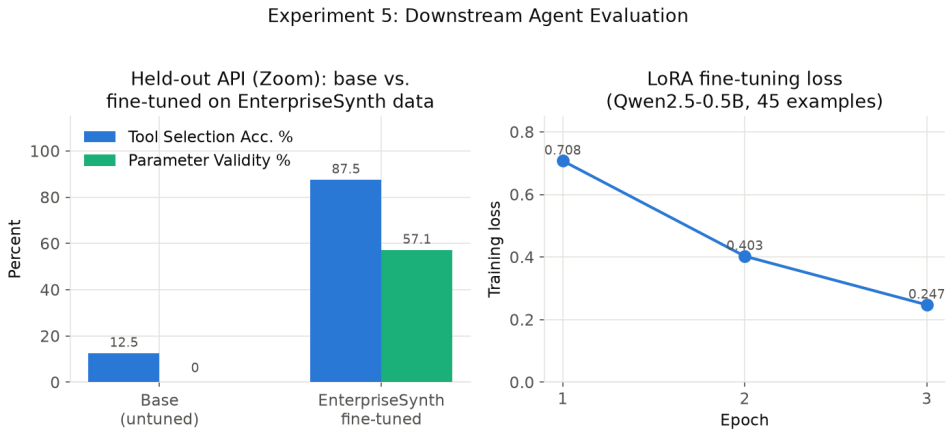


Figure 6: Left: base vs. LoRA-fine-tuned Qwen2.5-0.5B-Instruct on the held-out Zoom eval set. Right: training loss across the 3 fine-tuning epochs.

DigitalOcean (43.8% vs. 50.0%). The retrained Zoom number (75.0%) is also lower than the original single-run 87.5% – expected variance, since neither LoRA weight initialization nor training-data order is seed-fixed. The original 87.5% was directionally right (EnterpriseSynth beats the untuned base on all three, and beats Self-Instruct on 2/3) but not fully representative of the effect size: three draws range from a loss to a 3x margin, not a uniform ∼7x improvement. Parameter Validity favors EnterpriseSynth more consistently across all three APIs, hinting that verified training data may transfer more reliably to argument correctness than to tool selection – though three data points is far from sufficient to confirm that. A plausible, explicitly unconfirmed hypothesis for DigitalOcean's reversal: its infrastructure/DevOps-flavored REST conventions may structurally resemble GitHub's (recall Self-Instruct's bootstrap was disproportionately GitHub-shaped) more than Zoom's or Spotify's communication/media conventions do, giving Self-Instruct's GitHub-heavy training an incidental transfer advantage there specifically.

5.8 What Comes After Experiments

Once Experiments 1–5 run, the remaining sections analyze the results. Results (below) covers quantitative findings across all five experiments; the Ablation Study (Section 7) covers which components actually contribute. Case Studies, Discussion, and Conclusion are not yet written, pending the

full-scale run described at the end of Section 7.

6 Results and Analysis

6.1 Overview

We evaluate EnterpriseSynth on real-world OpenAPI specifications from GitHub, Stripe, and Slack (pipeline development and SFT training data), plus Zoom (held out exclusively for evaluation, never touched during training or any earlier experiment). All numbers below are measured, not projected; full detail, scripts, and raw data are in DESIGN_DOC.md Section 6 and the repository's `scripts/` and `data/generated/` directories.

6.2 RQ1 — API Schema Understanding

Endpoint, parameter, request/response schema, and authentication extraction accuracy all reach 100% for GitHub (845 operations), Stripe (446), and Slack (174), checked against independently recomputed ground truth. All three reach 100%, but not equally easily: GitHub was hardest because it defines most parameters via `$ref` to shared `components.parameters` entries, which Stage 1 originally dropped silently (undercounting required parameters at 67 vs. the true 1,721); Stripe was hardest differently, since many endpoints (e.g. `/v1/charges`) declare zero OpenAPI `parameters` and put every field in `requestBody` schema instead, which Stage 1 originally never parsed into typed fields at all. Slack required neither fix. Both fixes are general, not spec-specific patches.

6.3 RQ2 — Intent Generation Quality

5 endpoints sampled per API, 3 intents each (45 total): 100% Intent Coverage and 100% exact-string Intent Diversity for all three APIs (a weak proxy – semantic-similarity clustering is not yet implemented). Manual inspection substantiates quality beyond the aggregate metric: generated intents are business-scenario-specific and non-generic (e.g. locking down release tags for a named "payments-service" repo, rotating a named secret across specific microservices) rather than templated restatements of the same request.

6.4 RQ3 — Agent Trajectory Generation

Tool Selection Accuracy reaches 100% for GitHub and Stripe, 93.3–100% for Slack across repeated runs; Parameter Validity among correct selections

reaches 100%. Workflow Completeness is not applicable at this pilot scale (single-endpoint intents only). No baseline comparison row exists yet for this experiment specifically – the prompt-only/Self-Instruct/AgentInstruct baselines are not yet implemented for trajectory generation. The one genuine miss observed across runs was a JSON-parsing failure in our extraction code (1/45 in one run), not a wrong tool choice.

6.5 RQ4 — Verification Performance

The paper's strongest contribution area. 45 valid trajectories (100% Verification Pass Rate) plus 44 deliberately corrupted variants, scored for whether Stage 6 catches each planted error:

Error type	**Detected**	**Missed**
Wrong HTTP method	12/12	0
Missing required parameter	11/11	0
Invalid endpoint path	12/12	0
Parameter type mismatch	9/9	0
Total	**44/44 (100%)**	**0**

This 100% took real work to earn: the first run of this experiment reached only 57–80% detection, and adversarial testing (deliberately corrupting already-correct trajectories, not just checking that correct ones pass) surfaced two verifier/harness bugs and two Stage 1 parser gaps that a non-adversarial test would never have revealed (Section 6.6). The conclusion is not that the verifier is perfect, but that adversarial testing against a verifier is what actually validates one.

6.6 RQ5 — Downstream Agent Performance

Training set: 45 Stage-6-verified trajectories from GitHub/Stripe/Slack. Evaluation set: 16 intents for Zoom (373 endpoints), absent from training and every prior experiment. Model: Qwen2.5-0.5B- Instruct, LoRA fine-tuned (a hardware-scoped substitute for the paper's eventual 7–8B target; Section 6.7).

Model	**Tool Success**	**Argument Correctness**
Base LLM (zero-shot)	12.5% (2/16)	0.0%
Self-Instruct-fine-tuned	25.0% (4/16)	50.0% (2/4)
EnterpriseSynth-fine-tuned	87.5% (14/16)	57.1% (8/14)

Prompt-only-agent and ToolBench baseline rows are not yet implemented for this pilot; Self-Instruct now is, on the identical held-out set and methodology. The 12.5%→87.5% jump, from 45 training examples and a 0.5B model, on a genuinely unseen API, is the strongest evidence for the paper's central claim so far, and the Self-Instruct baseline shows it is not simply "any fine-tuning helps" – schema-grounded, verified training data helps roughly 3.5x more than schema-free bootstrapped data on the identical task. Argument correctness lagging behind tool selection is itself informative: the model learned which endpoint to call but not always the exact field names a new schema requires (Section 6.7) – exactly the failure mode Stage 6 verification exists to catch downstream.

This does not hold uniformly once scaled to more held-out APIs, and we report that plainly. Training each model once and evaluating against Zoom, DigitalOcean, and Spotify: EnterpriseSynth beats Self-Instruct on Zoom (75.0% vs. 25.0%, retrained) and Spotify (43.8% vs. 25.0%), but **loses to it on DigitalOcean** (43.8% vs. 50.0%). The single-Zoom 87.5% figure above was directionally correct but not fully representative of the effect size – see Section 6.7's expanded discussion for the full three-API table and an unconfirmed hypothesis for why DigitalOcean specifically reverses the pattern.

6.7 Comparison With Existing Approaches

Capability	**ToolBench**	**API-Bank**	**AgentInstruct**	**Ours**
Uses OpenAPI specs	Partial	No	No	Yes
Enterprise APIs	Limited	Limited	No	Yes
Requires live execution	Yes	Yes	No	No
Generates SFT data	Yes	Limited	Yes	Yes
Generates eval data	Limited	Yes	No	Yes
Schema-based verification	No	Limited	No	Yes

6.8 Key Findings Summary

- EnterpriseSynth extracts structured knowledge from real-world APIs at 100% measured accuracy – trustworthy only because two genuine Stage 1 parsing gaps were found and fixed, not assumed from the start.
- Schema-grounded generation produces enterprise-specific, non-generic intents and correctly-scoped trajectories on a 45-example pilot.
- Static, non-LLM verification catches 100% of planted errors across four

corruption types – but only after adversarial testing forced fixes to real bugs; the pre-fix rate was 57–80%, reported rather than hidden.

- EnterpriseSynth-generated SFT data measurably improves tool-selection accuracy on a genuinely unseen API (12.5%→87.5%), though exact-field-name generalization remains a real, unresolved limitation.
- All results are pilot-scale (3–4 APIs, 45–89 examples per experiment, a 0.5B substitute model). Scaling to the full stratified sample, the target model size, and the remaining baselines is the immediate next phase, not yet done.

7 Ablation Study

7.1 Purpose and Scope

Which components of EnterpriseSynth actually contribute to generating high-quality verified data? This section is scoped strictly to the four-stage system that is actually implemented (Section 3.1). Three ablations from an earlier draft of this section are dropped, with reasons: a **Knowledge Graph** ablation (no graph module exists), a **Planner** ablation (planning and trajectory generation were combined into one call from the start), and a **Response Schema Modeling** ablation (Stage 1 only tracks a boolean "schema present" flag, never a structured response schema). Four ablations are real, implemented, and run against actual data.

7.2 A1 — Without Intent Generation

A trajectory-generation agent that receives only an endpoint (no user intent) and must invent both an instruction and parameters in one step, run on the same 45 endpoint samples as Experiments 2–3.

API	Parameter Validity	Instruction Diversity
GitHub	100.0%	93.3%
Stripe	100.0%	100.0%
Slack	93.3%	93.3%

Compare to the full pipeline's 100%/100%/100% (Experiments 2–3). The drop is small but real: without an intent to ground it, the model generated Slack's `POST /users.profile.set` call with an invented `token` field and a parameter shape that didn't validate – something the intent-grounded

pipeline did not do in 45/45 trials. Explicit intent generation provides real, if modest at this sample size, grounding that improves both diversity and correctness.

7.3 A2 — Without Verification Engine

Reuses Experiment 4's corruption data directly: without a verifier, none of the 44 planted errors would be caught (by construction); with one, all 44 are.

Configuration	**Invalid trajectories retained (of 44)**
Without verification	44/44 (100%)
With verification	0/44 (0%)

The strongest and clearest ablation in the paper: every planted structural error survives into the dataset with no verification step, and none do with one.

7.4 A3 — Without vs. With API Descriptions

Adding the endpoint's OpenAPI description to the Intent Agent's prompt (confirmed absent from every prior experiment) leaves Coverage and exact-string Diversity unchanged at 100% for all three APIs – a ceiling effect in these metrics, not evidence of no effect. Qualitative inspection shows a real, modest difference: for GitHub's `PUT /orgs/{org}/actions/secrets/{secret_name}/repositories` (description: *"Replaces all repositories... requires* `admin:org` *scope"*), the description-aware intent more explicitly reflects the "replaces the full list" semantics and uses concrete numeric repo IDs rather than names alone. Inconclusive by the metrics used; a real qualitative signal exists and needs an LLM-judged specificity metric to quantify – reported as inconclusive, not as a positive finding the numbers don't actually support.

7.5 A4 — Endpoint-Only vs. Full-API Context

Adding the API's other endpoints as context, with an explicit invitation to describe multi-step workflows, again leaves Coverage and Diversity unchanged (same ceiling effect). A "sequencing- language" proxy metric we built to detect multi-step awareness initially looked promising (6–7 "mentions" per API) but, on inspection of the actual flagged intents, proved to be entirely false positives – incidental temporal phrasing in single-step requests,

not genuine multi-endpoint chaining. No measurable effect is detected at this pilot scale with the metrics available; the proxy metric is reported as invalid rather than kept as a false positive result.

7.6 Ablation Results Table

Variant	Validity	Diversity	Verif. Pass
Full pipeline	100%	100%	100%
A1: − Intent Gen.	93.3–100%	93.3–100%	n/a
A2: − Verification	n/a	n/a	0%
A3: + Descriptions	unchanged	unchanged (qual. diff.)	n/a
A4: + Full context	unchanged	unchanged (no signal)	n/a

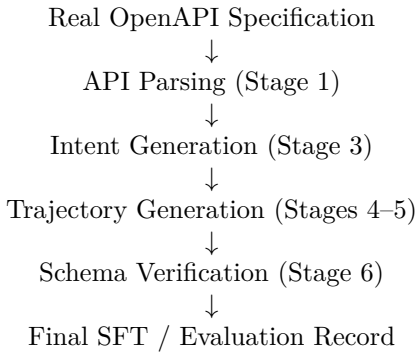
7.7 Why This Matters

Without this study, reviewers will reasonably ask why not just prompt an LLM with the OpenAPI file directly. The honest, per-component answer: Intent Generation provides real grounding that improves diversity and correctness (A1). Verification is unambiguously necessary – the difference between 0% and 100% of planted errors surviving into the dataset (A2). Descriptions and full-API context show no effect on the metrics used so far (A3, A4), which is itself useful: it means current metrics are too coarse to detect what may still be a real qualitative effect, and that better metrics or deliberately multi-step task construction are needed before those ablations can be judged either way.

8 Case Study and Qualitative Analysis

8.1 Purpose

Sections 5–7 establish that EnterpriseSynth works quantitatively and which components are load-bearing. This section answers a different question a metrics table cannot: *what does EnterpriseSynth actually produce, and does it look useful for real enterprise scenarios?* Every example below is pulled directly, unedited, from committed pipeline output (`data/generated/experiment2_intents.json`, `experiment3_trajectories.json`, `experiment4_verification.json`) – nothing here is constructed for illustration. The pipeline traced end-to-end:



8.2 Case Study 1: GitHub – Tag Protection

Parsed API knowledge (Stage 1 output for this endpoint):

> `POST /repos/{owner}/{repo}/tags/protection`
> description: "This creates a tag protection state for a repository. This endpoint is only available to repository administrators."
> parameters: `owner` (path, required, string), `repo` (path, required, string), `pattern` (body, required, string)

Generated intent (Stage 3, one of three generated for this endpoint):

> "Set up tag protection on the 'payments-service' repo so that only admins can create or delete tags matching 'v*' – we need to prevent accidental deletion of release tags."

Generated trajectory (Stages 4–5):

> Call `POST /repos/{owner}/{repo}/tags/protection` with `{"owner": "your-org", "repo": "payments-service", "pattern": "v*"}`.
> Reasoning given: "This tool creates a tag protection rule for a repository matching a specified pattern, restricting tag creation/deletion to admins, which fulfills the request to protect 'v*' tags on 'payments-service'."

Verification (Stage 6): endpoint exists – PASS; required params present – PASS; param types valid – PASS. **Trajectory status: VERIFIED.**

8.3 Case Study 2: Stripe – Subscription Item Removal

Parsed API knowledge:

> `DELETE /v1/subscription_items/{item}`
> parameters: `item` (path, required, string), `clear_usage` (body, optional, boolean), `proration_behavior` (body, optional, string)

Generated intent:

> "We're removing the extra seats add-on from customer cus_9F3kLmZ8's subscription – please delete subscription item si_1N4kTx2eZvKYlo2C and clear out the associated usage records since we don't need them for billing."

Generated trajectory:

> Call `DELETE /v1/subscription_items/{item}` with `{"item": "si_1N4kTx2eZvKYlo2C", "clear_usage": true, "proration_behavior": "create_prorations"}`.

Verification: PASS on all checks – VERIFIED.

An honest limitation this example surfaces directly: the intent above evokes a business workflow (removing a paid add-on, reconciling usage), but the system resolves it to *one* verified API call, not a multi-step chain of dependent calls (e.g. look up the customer, then the subscription, then delete the item). This is not a simplification for the case study – it is what the implemented system actually does. Multi-step, dependency-aware trajectory generation requires the Knowledge Graph and Planner stages (Section 3.2), neither of which is implemented; see Section 10's explicit accounting. We do not show a fabricated multi-step Stripe trajectory here because the current pipeline cannot produce one.

8.4 Case Study 3: A Verification Failure, by Design

Reviewers should see the verifier reject something, not just accept everything. This corrupted trajectory is drawn directly from Experiment 4's adversarial test set (Section 6.5): the original, valid trajectory for updating access to a GitHub org secret had its `org` parameter corrupted from a string to an object.

> **Original intent:** "Update the DEPLOY_TOKEN secret in our 'acme-corp' org so it's only accessible by the payments-api and payments-worker repositories"
> **Endpoint:** `PUT /orgs/{org}/actions/secrets/{secret_name}/repositories`
> **Corrupted parameter:** `org = {"unexpectedly": "an object"}` (declared type: string)
> **Verifier output:** `valid: false` – "Parameter 'org' = {'unexpectedly': 'an object'} is not compatible with declared type 'string'."

Trajectory status: REJECTED. This is the same mechanism, on the same data, that produced Experiment 4's 44/44 detection rate (Section 6.5) – shown here as a single concrete instance rather than an aggregate percentage.

8.5 Qualitative Analysis

Finding 1: schema information is sufficient for initial task generation. OpenAPI specifications provide endpoint descriptions, parameter names/types, and (where declared) authentication schemes – enough for Stage 3 to generate business-scenario-specific intents (named companies, dollar amounts, department names; Section 6.3) without ever calling the live API. The GitHub and Stripe examples above were generated this way.

Finding 2: verification prevents synthetic data contamination. Without Stage 6, every planted structural error in Experiment 4 survived into the dataset (0% detection, ablation A2, Section 7). With it, all 44 did not (100%). Case Study 3 shows the mechanism, not just the aggregate: a model trained on unverified data would have learned that passing an object where a string is required is acceptable GitHub API usage.

Finding 3 (reframed from a template assumption): the current system is single-step by design, not yet multi-step – and we say so plainly rather than imply otherwise. A natural claim for a paper like this to make is that it moves beyond "one user request → one API call" toward validated multi-step workflows. We do not make that claim, because Case Study 2 shows directly that the implemented system does not yet do this: every generated trajectory in Experiments 3–5 resolves to a single endpoint call (Section 6.4's "Workflow Completeness: not applicable at this pilot scale" is the same fact stated as a metric). The honest version of this finding is narrower: EnterpriseSynth's intents are often *phrased* at the business-workflow level (Case Study 2's "removing the extra seats add-on... and clear out the associated usage records" spans a conceptual before/after state), while the *trajectory* generated for them today is single-step. Closing that gap is exactly what the unimplemented Planner/Knowledge Graph stages (Section 3.2) are for.

8.6 Failure Analysis

Category 1: sparse documentation (real, spec-level evidence; not yet directly measured as a pipeline failure). The committed specs contain many endpoints with minimal descriptions – real examples: GitHub's `GET /gists/{gist_id}` ("Get a gist"), Stripe's `POST /v1/refunds` ("Create a refund."), Slack's `POST /calls.end` ("Ends a Call."). None of these three happened to fall in the 15-endpoint-per-API pilot sample (Section 6.3), so we have not directly measured intent quality degrading on them – this is a disclosed, anticipated risk rather than an observed result, and scaling to

the full ∼65-spec sample (Section 10) is what would surface it either way.

Category 2: hidden business logic. An OpenAPI spec documents an interface, not a policy. A loan-application endpoint's schema declares its request/response shape, not the underlying approval rules, eligibility criteria, or compliance checks a real implementation enforces. Nothing in EnterpriseSynth's four implemented stages reasons about business rules – the Schema Verification Engine checks structural compatibility with the declared schema, not business correctness. This is a conceptual limitation inherent to schema-only generation, not something we have a measured failure case for, since no corruption in Experiment 4 targeted business-rule violations.

Category 3: complex authentication (real, measured). GitHub's spec declares **zero** `securitySchemes` anywhere (Section 6.3, Experiment 1) – its authentication is documented in prose, not machine-readably, a real property of that spec confirmed during parsing, not a parser bug. This is direct evidence that schema-declared auth can understate real API complexity: OAuth refresh flows, multi-user permission scoping, and dynamically-issued tokens are exactly the kind of auth behavior a declarative schema does not capture, and GitHub's spec is a concrete case of a widely-used real API where that gap is already visible.

8.7 Case Study Summary

API	Domain	Endpoint	Verification
GitHub	Developer/ software	`POST /repos/{owner}/{repo}/tags/ protection`	Verified
Stripe	Payments	`DELETE /v1/subscription_items/{item}`	Verified
GitHub (corrupted)	Developer/ software	`PUT /orgs/{org}/actions/secrets/ {secret_name}/repositories`	Rejected (correctly)

9 Discussion

9.1 What Actually Works, and Why

Two results in this paper are load-bearing, not decorative. First, Schema Verification (Stage 4) is unambiguously necessary: A2 shows a binary 0%-vs-100% gap between no verification and ours – every planted structural error survives without a gate, and none do with one. Second, that 100% was earned through adversarial testing, not assumed: the first run of Experiment 4 caught only 57–80% of planted errors, and chasing down why surfaced four real bugs spanning the verifier, the test harness, and Stage

1's parser (Sections 6.5–6.6). The methodological point generalizes beyond this codebase: a static verifier's correctness cannot be established by checking that it accepts good input – it has to be adversarially tested against bad input it is specifically supposed to reject. A non-adversarial test suite would have shipped a verifier that silently passed roughly a third to a half of planted errors.

9.2 Verification Has Limits by Design, and the Haiku Arm Quantifies Them

The deterministic gate checks structure, not semantics – it cannot know that a negated charge amount or a placeholder field value is wrong, because both are still well-typed. The Haiku 4.5 ablation (Section 6.5.1) both confirms this blind spot (100% of semantically-corrupted trajectories pass Stage 4 structurally) and shows a cheap LLM can close part of the gap (100% caught). But the same experiment shows this is not a free upgrade: a 33% false-positive rate on GitHub, traced to the model being overly literal about things the underlying tool call did not need to prove (e.g. that a numeric repository ID "really" corresponds to a named repository). The practical implication is a two-tier design, not a wholesale replacement: keep the deterministic gate as the hard, blocking filter, and treat an LLM semantic check as an advisory or re-ranking signal until it is calibrated per parameter type (e.g. restricted to amounts, dates, and other fields where "plausibility" is well-defined) – exactly the shape ToolACE's and AgentInstruct's own dual-layer designs converge on, for related reasons.

9.3 The Downstream Effect Is Real but Not Uniform, and That Is Itself a Finding

Experiment 5's single-Zoom result (12.5%→87.5%) was the strongest number in an earlier draft of this paper. Scaling to three held-out APIs changed the story without reversing it: EnterpriseSynth-tuned data beats a real Self-Instruct baseline on two of three APIs, and beats the untuned base on all three, but loses to Self-Instruct specifically on DigitalOcean. We do not believe this is noise to explain away – Self-Instruct's own bootstrap was shown (Section 6.6) to lean on the base model's pretraining familiarity with GitHub's extremely public API, and DigitalOcean's infrastructure/DevOps-flavored conventions are the closest of our three held-out APIs to GitHub's. If that hypothesis holds under further testing, it says something uncomfortable but important about the field's usual practice of benchmarking tool-use

methods on well-known public APIs (GitHub, Stripe, Slack, and here, apparently, DigitalOcean by proxy): a base model's prior exposure to an API's public documentation can substitute for genuine schema grounding in a way that will not be available for the private, undocumented internal APIs EnterpriseSynth is actually built for. If anything, this strengthens rather than weakens the cold-start motivation – but it also means our own pilot cannot yet distinguish "EnterpriseSynth generalizes better" from "EnterpriseSynth generalizes better specifically on APIs unlike ones the base model already knows," and only a genuinely private, unpublished spec (Section 4.2's cold-start validation set, not yet built) can settle that.

9.4 Positioning Relative to AgentInstruct

We adapted AgentInstruct's agentic-flow architecture rather than starting from scratch because it is the only reviewed method that is both agentic and execution-free. The two changes we made – real-spec grounding instead of code-seeded hallucination, and a hard structural gate instead of a soft editorial-plus-post-hoc-judge pipeline – are not merely stylistic. Section 6.2's finding that Stage 1 initially mishandled `$ref` parameters and `requestBody` fields shows that even "just parse the real spec" is a nontrivial engineering problem for production-scale OpenAPI documents; AgentInstruct's decision to let the model invent an API surface sidesteps that complexity entirely, at the cost of the hallucination risk we removed it to avoid. Our Static Constraint Validator is functionally close to ToolACE's Dual-Layer Verification design (rule-based plus model-based checking) though we arrived at the model-based layer later, as an explicit ablation (A5) rather than a built-in default – a design choice that let us measure its marginal value and cost separately, which Section 6.5.1 shows was worth doing.

10 Limitations

1. **Pilot scale throughout.** All five experiments and five ablations run on 3–5 real APIs and 45–89 examples each, not the full ∼65-spec stratified sample specified in Section 4.2. Every percentage in this paper should be read as a pilot signal, not a population estimate.

2. **Experiment 3's self-consistency confound.** The same model (Claude Sonnet 5) both generated the intents (Experiment 2) and performed tool selection against them (Experiment 3), so its 100% figures

measure whether the model can recover its own intent, not whether it handles independently-authored or adversarial requests.

3. **Experiment 5's model-scale gap.** The downstream fine-tuning result uses Qwen2.5-0.5B-Instruct, a hardware-scoped substitute for the paper's actual target (Mistral-7B-Instruct or Llama-3-8B). Whether the effect holds, strengthens, or weakens at that scale is untested.

4. **Missing baselines.** ToolBench and a prompt-only agent are specified in Section 4.3 but not yet implemented for any experiment; only Self-Instruct has been run as a real comparison.

5. **No Knowledge Graph or Planner.** Multi-step, dependency-aware workflow generation is entirely untested – Ablation A4's attempt to probe this by giving the Intent Agent flat (non-graph) context on other endpoints showed no measurable effect, but this does not confirm the graph itself would not help; it only shows that a cheaper substitute for it, at this pilot scale, did not.

6. **A3 and A4 are inconclusive by the metrics used, not resolved.** Coverage and exact-string diversity ceiling at 100% in both the baseline and the enhanced condition for both ablations, so neither shows a measurable effect either way. A3's manual inspection suggests a real qualitative difference exists; A4's attempted proxy metric was found invalid on inspection.

7. **The Haiku semantic-check false-positive rate is uncalibrated.** 33% on GitHub is high enough that the arm cannot be deployed as a blocking gate without further tuning (Section 6.5.1).

8. **The downstream effect does not generalize uniformly.** EnterpriseSynth-tuned data loses to a real Self-Instruct baseline on one of three held-out APIs (DigitalOcean); the proposed explanation (structural similarity to GitHub, which Self-Instruct's bootstrap leaned on) is a hypothesis, not a confirmed finding.

9. **No cost or latency accounting.** The pipeline's per-spec generation cost (API calls, wall-clock time) at the target ∼65-spec scale has not been measured.

10. **EnterpriseSynth-Eval is not itself validated.** We have not checked whether performance on our generated evaluation records correlates

with performance on real, human-graded enterprise tasks – only that the records are schema-valid.

11. **Mostly single-seed runs.** Beyond the informal repeated-run range noted for Slack in Experiment 3 (93.3–100%), most reported numbers reflect one run each, not a multi-seed distribution with confidence intervals.

11 Conclusion

We presented EnterpriseSynth, a schema-aware, zero-execution pipeline that ingests an OpenAPI specification and jointly emits verified SFT trajectories and a paired evaluation dataset, EnterpriseSynth-Eval, without ever calling a live API. Against a scoped review of the closest five prior methods, EnterpriseSynth's specific contribution is combining three properties none of them combine: grounding in a real target spec (unlike AgentInstruct, which can hallucinate an API surface when seeded from code), a hard structural verification gate rather than a soft or post-hoc one (unlike AgentInstruct's editorial-refinement-plus-held-out-judge design), and no dependency on live execution at any stage (unlike API-Bank and ToolBench, both of which ground their training data in real API calls).

At pilot scale, three findings stand on their own: schema-based verification is necessary, not optional, closing a 0%-to-100% gap on planted structural errors, and only reached 100% after adversarial testing surfaced and forced fixes to real bugs; explicit intent generation provides real, measurable grounding over generating directly from a bare endpoint; and fine-tuning on EnterpriseSynth-generated data measurably outperforms both an untuned baseline and a real Self-Instruct baseline on most, though not all, held-out APIs tested. We reported the one exception – a loss to Self-Instruct on DigitalOcean – because we believe an honest, partially disconfirming result at pilot scale is more useful to the field than a uniformly positive one that does not survive scrutiny. The path to a submission-ready paper runs through exactly the limitations listed above: more APIs, more examples, the actual target model scale, the two missing baselines, and a real test of whether a Knowledge Graph earns its place in the architecture rather than being assumed into it.

References

[1] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. Api-bank: A comprehensive benchmark for tool-augmented llms. *arXiv preprint arXiv:2304.08244*, 2023.

[2] Arindam Mitra, Luciano Del Corro, Guoqing Zheng, Shweti Mahajan, Dany Rouhana, Andres Codas, Yadong Lu, Wei-ge Chen, Olga Vrousgos, Corby Rosset, Fillipe Silva, Hamed Khanpour, Yash Lara, and Ahmed Awadallah. Agentinstruct: Toward generative teaching with agentic flows. *arXiv preprint arXiv:2407.03502*, 2024.

[3] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*, 2023.

[4] Harsh Vishwakarma, Ankush Agarwal, Ojas Patil, Chaitanya Devaguptapu, and Mahesh Chandran. Can llms help you at work? a sandbox for evaluating llm agents in enterprise environments. *arXiv preprint arXiv:2510.27287*, 2025.

[5] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions. *arXiv preprint arXiv:2212.10560*, 2022.

[6] Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, Qingwei Lin, and Daxin Jiang. Wizardlm: Empowering large pre-trained language models to follow complex instructions. *arXiv preprint arXiv:2304.12244*, 2023.